

REGRAS OFICIAIS DE BRANCHES E COMMITS

Marketplace Comunitário Raízes do Araguaia

1. Objetivo do Documento

Este documento define o padrão oficial de uso de branches e commits da equipe.

O objetivo é:

- evitar conflitos
- organizar o GitHub
- manter histórico limpo
- facilitar revisões
- melhorar rastreabilidade
- impedir perda de código
- padronizar entregas

Todos os membros devem seguir estas regras obrigatoriamente.

2. Regra Principal

NUNCA DESENVOLVER DIRETAMENTE NA MAIN

A branch `main` representa:

- versão estável
- versão oficial
- código validado

É proibido:

- commitar diretamente na `main`
 - desenvolver diretamente nela
 - enviar código sem Pull Request
-

3. Estrutura Oficial de Branches

Toda funcionalidade deve possuir sua própria branch.

Formato obrigatório:

``txt id="i0ydp5" feat/nome-da-funcionalidade

4. Padrões Oficiais de Branches

Funcionalidades

``txt id="zq5l2z"

feat/home-banner

feat/map-filters

feat/product-upload

feat/user-progress

Correções

``txt id="9k7d4h" fix/home-navigation fix/map-location fix/catalog-loading

Refatoração

``txt id="tr9m1f"

refactor/catalog-hooks

refactor/shared-components

Documentação

``txt id="7c7r8x" docs/manual-geral docs/github-flow

5. Branches Oficiais da Equipe

Guilherme

``txt id="8v7lfr"

feat/guilherme-home

Flavio

``txt id="h7w5ga" feat/flavio-mapa

```
---
```

```
## Julia
```

```
```txt id="6kl3qz"  
feat/julia-escolinha
```

---

## Cecilia

```
```txt id="m6zp2g" feat/cecilia-catalogo
```

```
---
```

```
## Marianne
```

```
```txt id="5vz91x"  
feat/marianne-banco
```

---

## Henrique

```
```txt id="c7h2xk" feat/henrique-infra
```

```
---
```

```
# 6. Processo Oficial Para Criar Branch
```

```
Antes de criar qualquer branch:
```

```
```bash id="kcb1om"  
git checkout main
git pull origin main
```

Depois:

```
```bash id="3ocjxt" git checkout -b feat/nome-da-funcionalidade
```

```
---
```

```
# 7. Regras Oficiais de Commit
```

```
Os commits devem ser:
```

- pequenos
- claros
- organizados
- específicos

Evitar:

- commits gigantes
- commits genéricos
- múltiplas funcionalidades no mesmo commit

8. Prefixos Oficiais de Commit

Funcionalidade Nova

```
```txt id="h8ij3l"
feat:
```

Exemplo:

```
```bash id="n7e1kq" git commit -m "feat: adiciona cards da home"
```

Correção de Bug

```
```txt id="m4p5zq"
fix:
```

Exemplo:

```
```bash id="u4s8po" git commit -m "fix: corrige filtro do mapa"
```

Refatoração

```
```txt id="4o9h4v"
refactor:
```

Exemplo:

```
```bash id="h9x0tu" git commit -m "refactor: reorganiza hooks do catalogo"
```

```
---

## Documentação

```txt id="8y1qoo"
docs:
```

Exemplo:

```
```bash id="q3o5kt" git commit -m "docs: adiciona manual de desenvolvimento"
```

```
---

## Estilo Visual

```txt id="m7s5dz"
style:
```

Exemplo:

```
```bash id="8c0oim" git commit -m "style: ajusta responsividade da home"
```

```
---

# 9. O Que NÃO Fazer nos Commits

Proibido:

❌ commit
❌ teste
❌ update
❌ mudanças
❌ ajuste
❌ vários arquivos
❌ commit final

Os commits devem explicar claramente o que foi feito.

---

# 10. Frequência Ideal de Commits

Recomendação:

- fazer commits pequenos
- não acumular muitas alterações
- commitar após blocos funcionais prontos
```

Exemplo:

- criou componente → commit
- corrigiu bug → commit
- terminou responsividade → commit

11. Atualização da Branch

Antes de abrir Pull Request:

```
``bash id="d2n9d1"
git checkout main
git pull origin main
git checkout sua-branch
git merge main
```

Resolver conflitos localmente antes do PR.

12. Regras Oficiais de Pull Request

Todo Pull Request deve:

- possuir descrição
- explicar impacto
- informar como testar
- indicar dependências
- estar testado

Nenhum PR deve ser aberto:

- com erro
- sem build
- sem lint
- sem teste mínimo

13. Arquivos de Alto Risco

Mudanças nesses arquivos exigem comunicação obrigatória:

```
txt id="n7d3ey"
frontend/src/main.tsx
frontend/src/routes/router.tsx
frontend/package.json
```

```
frontend/src/shared/  
frontend/src/index.css
```

14. Regras de Merge

Merge permitido apenas quando:

- build funcionando
 - lint funcionando
 - revisão concluída
 - conflitos resolvidos
 - PR aprovado
-

15. Objetivo Final

Essas regras existem para garantir:

- organização
- estabilidade
- colaboração eficiente
- histórico limpo
- rastreabilidade
- segurança do projeto
- desenvolvimento escalável

Todos os membros devem seguir este padrão durante toda a evolução da plataforma.